Scenario-Based Compositional Statistical Model Checking for Safety Specifications

Abhinav Pomalapally, Arya Raeesi,
Kevin Kai-Chun Chang, Beyazit Yalcinkaya, and Sanjit A. Seshia

University of California, Berkeley, USA
{abhinav_pomalapally,aryaraeesi}@berkeley.edu
{kaichunchang,beyazit,sseshia}@berkeley.edu

Abstract. In safety-critical domains such as autonomous driving, systems must be evaluated across a large number of environment conditions, often represented as *composite scenarios* built from *primitive scenarios*. Existing statistical model checking (SMC) approaches analyze each composite scenario independently, requiring many expensive simulations and resulting in substantial redundant computation when scenarios share common structure. This work introduces a scenario-based compositional SMC framework for safety and co-safety specifications, enabling efficient analysis of composite scenarios. Our approach decomposes scenarios into primitives and specifications into sub-specifications, verifies each primitive independently, and composes the resulting statistical estimates using importance sampling and kernel density estimation. Our empirical evaluation shows that the proposed framework can accurately answer verification queries for previously unseen composite scenarios while reducing simulation cost through parallelization and trace reuse.

Keywords: Statistical Model Checking · Compositional Verification · Scenario-Based Testing · Safety Specifications · Runtime Verification

1 Introduction

Artificial intelligence (AI) systems are increasingly deployed in safety-critical domains, including autonomous driving, robotics, healthcare, and cyber-physical systems in general. As these systems become more pervasive, ensuring their safety and reliability remains a fundamental challenge. However, the complexity of modern AI-based systems renders traditional verification techniques largely inapplicable. Classical formal verification methods typically require an explicit and tractable model of the system under analysis, an assumption that is often violated in contemporary AI systems due to their inherent opacity.

A practical alternative is to treat the system under analysis as a black box and reason about its behavior through simulation-based analysis [20]. This approach is particularly prevalent in autonomous driving, where validating a system through exhaustive real-world testing is infeasible. Instead, developers evaluate autonomous driving systems using a corpus of *scenarios*, either manually

designed by domain experts or extracted from real-world driving data, and assess the system's performance across these scenarios through repeated simulations.

Statistical model checking (SMC) provides a principled framework for such simulation-based verification [16]. SMC is a formal verification technique that determines whether a stochastic system satisfies a given property within a particular scenario by analyzing a finite set of execution traces using statistical methods such as hypothesis testing or confidence interval estimation. By mitigating the state-space explosion associated with exhaustive model checking, SMC enables verification of systems whose complexity would otherwise make formal analysis intractable. Nevertheless, SMC continues to face significant scalability challenges. Obtaining meaningful and high-confidence verification results often requires executing a large number of computationally expensive, long-running simulations. Moreover, SMC must typically be repeated whenever the scenario defining the environment conditions changes, leading to redundant computation.

In this paper, we introduce a scenario-based compositional SMC framework for safety and co-safety specifications, i.e., specifications whose violation or satisfaction can be determined from finite executions. Our approach leverages both scenario and specification decomposition. Given a *composite scenario*, we first decompose it into a collection of *primitive scenarios*. Simultaneously, we decompose the target safety specification into a set of sub-specifications. The black-box system is then evaluated independently on each primitive scenario with respect to all relevant sub-specifications. These verification tasks can be executed in parallel and require shorter simulation runs than the original monolithic verification problem. Finally, we compose the resulting statistical estimates to obtain a verification result for the original composite scenario and specification. Furthermore, our compositional framework enables the reuse of traces generated for primitive scenarios across multiple composite scenarios, thereby avoiding redundant simulation effort and significantly reducing overall verification cost.

We summarize our contributions as follows: (i) we introduce a novel compositional statistical model checking framework for safety and co-safety specifications; (ii) we implement the proposed framework and integrate it into VerifAI [9], a toolkit for the formal design and analysis of AI-based systems; and (iii) we conduct an empirical evaluation of the proposed approach and demonstrate its effectiveness in improving verification efficiency while maintaining accuracy.

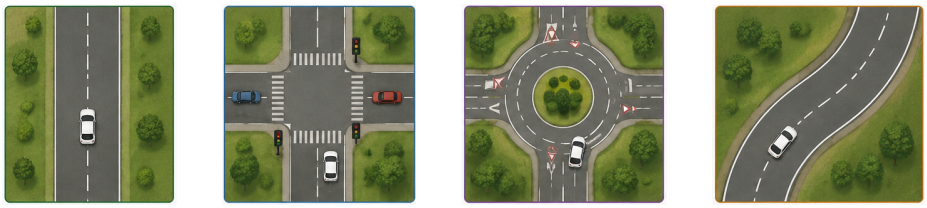
Related Work. To address the verification of AI-based systems, simulation-based formal analysis has been widely adopted [20,9,1]. Specifically, statistical model checking (SMC) [18,19,25,26,16,5,27,2] mitigates the state-space explosion of exact formal methods by evaluating a finite set of execution traces, providing probabilistic guarantees bounded by statistical confidence intervals. Seshia et al. [20] advocate for a compositional approach to scale up verification of AI systems. Classic compositional verification (e.g., [6,7,4,14,22]) is not directly applicable to simulation-based verification of AI systems. Existing compositional approaches for AI systems vary in their axes of decomposition; while some frameworks decompose internal system architectures or neural network layers [21,10],

our work focuses on the spatial and temporal decomposition of environmental scenarios and specifications.

Dreossi et al. [8] has proposed a framework combining compositional analysis with simulation-based verification, but it focuses strictly on component-level decomposition and targets active falsification rather than SMC. The closest prior work to the proposed framework introduces a compositional approach to SMC [24]. However, it is strictly restricted to Markovian, i.e., memoryless, specifications. Our new framework extends compositional SMC to non-Markovian safety and co-safety specifications, overcoming a fundamental limitation.

Another related line of work explores scenario decomposition, where environments and specifications are divided into primitive segments either spatially [23,13] or temporally [3]. However, these differ significantly in their verification methods: they rely on explicit white-box access [13], offline probabilistic abstractions [23], or remain confined to falsification [3]. In contrast, our framework achieves black-box compositional SMC for safety and co-safety properties.

2 Motivating Example



(a) `S` (b) `X` (c) `O` (d) `C`

Fig. 1: Primitive scenarios considered throughout the paper: a straight road segment (`S`), an intersection (`X`), a roundabout (`O`), and a curved road segment (`C`).

Consider evaluating an autonomous vehicle across various driving scenarios (Figure 1). A common approach to constructing such scenarios is to use a set of simpler primitive scenarios as building blocks for defining more complex composite scenarios. Suppose that our primitive scenarios consist of a straight road segment (`S`), an intersection (`X`), a roundabout (`O`), and a curved road segment (`C`). Scenario description languages such as Scenic [11,12] support this style of programmatic scenario construction through several composition operators. *Sequential composition* specifies the exact order in which scenarios are encountered (e.g., $\texttt{S} \rightarrow \texttt{X}$). *Choice composition* randomly selects one scenario from a set of alternatives (e.g., $\texttt{choose}(\texttt{S}, \texttt{X}, \texttt{O}, \texttt{C})$). Finally, *shuffle composition* randomly permutes the order of a collection of scenarios (e.g., $\texttt{shuffle}(\texttt{S}, \texttt{X}, \texttt{O}, \texttt{C})$).

Our goal is to efficiently test the autonomous vehicle in composite scenarios against safety and co-safety specifications, i.e., temporal specifications whose satisfaction or violation can be determined from a finite execution. A monolithic application of SMC treats each composite scenario independently. For every new

scenario and specification pair, SMC executes a large number of simulations and analyzes the resulting traces. As the complexity of composite scenarios grows, this process quickly becomes computationally expensive, requiring repeated execution of long-running simulations that often share substantial structure.

Our approach avoids this redundancy by shifting the computational effort to the primitive scenarios. Instead of directly evaluating every composite scenario, we execute the system independently within each primitive scenario and store the resulting traces. We then evaluate these traces against all sub-specifications induced by the target specification, corresponding to the sub-automata of its monitor automaton. Given a composite scenario, we compose the previously computed results to estimate the probability of satisfying the target specification.

A key challenge is that the distribution of behaviors observed in an isolated primitive scenario may differ from the distribution induced when that same primitive appears as part of a larger composite scenario. To account for this distributional mismatch, we employ importance sampling [15] together with kernel density estimation (KDE). Intuitively, traces generated from primitive scenarios are weighted according to their likelihood under the target composite scenario. This enables us to estimate results for previously unseen composite scenarios without actually running the system in those scenarios. Consequently, our approach generates computationally efficient and highly accurate estimates for SMC queries.

3 Method

In this section, we present our compositional approach for checking safety specifications. We assume that the controller under consideration is Markovian, i.e., it outputs actions that solely depend on the current state of the plant, and that it operates in a Markov Decision Process (MDP). A *primitive scenario* defines environment conditions from a set of initial states to a set of terminal states, e.g., the primitive scenario in Figure 1b starts at the car's initial position and ends when the car leaves the intersection. A *composite scenario* is a composition of such primitives, e.g., $\texttt{S} \rightarrow \texttt{X}$ sequentially composes `S` and `X`, defining a scenario that starts at the beginning of a straight road and ends after an intersection.

Let $\tau \sim (\pi, \mathcal{S})$ denote a (labeled) trace sampled by running a policy π in a scenario $\mathcal{S}$, let $\mathcal{T}$ be a finite set of such traces, and let ϕ be a safety specification in automaton form. SMC estimates the specification satisfaction probability as:

$$\Pr[\tau \models \phi] = \frac{1}{|\mathcal{T}|} \sum_{\tau_i \in \mathcal{T}} \mathbb{1}\{\tau_i \models \phi\}, \tag{1}$$

where $\tau \models \phi$ denotes that the trace τ satisfies the safety specification ϕ. Note that $|\mathcal{T}|$ can be chosen via Hoeffding's inequality to achieve a desired confidence level.

We rewrite this expression in terms of the *state distribution* of the safety specification. Observe that for a safety specification $\phi = (Q, \Sigma, \delta, q_0, F)$, its state distribution $\mu_0 \in \Delta(Q)$ is initially concentrated at q_0, i.e., $\mu_0(q_0) = 1$,

as q_0 is the initial state. Let $\mu_{\mathcal{S}}$ denote the state distribution induced by transitioning the automaton using traces $\tau \sim (\pi, \mathcal{S})$. $\mu_{\mathcal{S}}$ summarizes execution history within each trace relevant to the specification, allowing our method to handle non-Markovian temporal properties through Markovian updates over automaton states. Therefore, we can rewrite Equation (1) as follows:

$$\Pr[\tau \models \phi] = \sum_{q \in Q} \mathbb{1}\{\delta^*(q, \tau) \in F\}\mu_0(q) = \sum_{q \in Q} \mathbb{1}\{q \in F\}\mu_{\mathcal{S}}(q), \tag{2}$$

where δ^* is the transitive closure of the automaton transition function δ. Observe that the expression in the middle marginalizes over all initial states using the initial state distribution μ_0, and the expression on the right marginalizes over all final states using the resulting output state distribution $\mu_{\mathcal{S}}$. Let $\mathcal{T}_{q \to q'}$ denote the subset of the given traces $\mathcal{T}$ that ends at an automaton state q' starting from an automaton state q. We can estimate $\mu_{\mathcal{S}}$ using μ_0 and $\mathcal{T}$ as follows:

$$\mu_{\mathcal{S}}(q') = \sum_{q \in Q} \mu_0(q) \frac{1}{|\mathcal{T}_{q \to q'}|} \sum_{\tau_i \in \mathcal{T}_{q \to q'}} \mathbb{1}\{\delta^*(q, \tau_i) = q'\}. \tag{3}$$

This expression shows the one-step propagation rule for computing the resulting automaton state distribution after a scenario $\mathcal{S}$. We emphasize that Equation (3) is not intended as a novel derivation; rather, it provides the empirical estimation needed to obtain the automaton state distribution induced by a scenario from the initial distribution and available traces. We use this idea to compositionally compute SMC queries for sequential, choose, and shuffle compositions.

As co-safety specifications are the complement of safety specifications, given a co-safety specification ϕ, we take its complement $\bar{\phi}$ which is a safety specification, run SMC, and take the complement of the resulting satisfaction probability, i.e.,

$$\Pr[\tau \models \phi] = 1 - \Pr[\tau \models \bar{\phi}], \tag{4}$$

where $\bar{\phi}$ is a safety specification, and $\Pr[\tau \models \bar{\phi}]$ is computed as in Equation (1).

3.1 Sequential Composition

Consider the sequential composition $\mathtt{S} \to \mathtt{X}$. Let $\mathcal{T}^{\mathtt{S}}$ and $\mathcal{T}^{\mathtt{X}}$ denote traces independently generated for the primitive scenarios $\mathtt{S}$ and $\mathtt{X}$, respectively. Our goal is to use the primitive scenario traces $\mathcal{T}^{\mathtt{S}}$ and $\mathcal{T}^{\mathtt{X}}$ to estimate the satisfaction probability for the composite scenario $\mathtt{S} \to \mathtt{X}$ without actually generating traces for it. Let $\tau^{\mathtt{S}\to\mathtt{X}}$, $\tau^{\mathtt{S}}$, and $\tau^{\mathtt{X}}$ denote traces sampled for $\mathtt{S} \to \mathtt{X}$, $\mathtt{S}$, and $\mathtt{X}$, respectively. Let μ_0 be the initial automaton state distributions, and let $\mu_{\mathtt{S}\to\mathtt{X}}$, $\mu_{\mathtt{S}}$, and $\mu_{\mathtt{X}}$ denote the automaton state distribution induced by the scenarios $\mathtt{S} \to \mathtt{X}$, $\mathtt{S}$, and $\mathtt{X}$, respectively. Using Equation (2), we can compute the SMC query as follows:

$$\Pr[\tau^{\mathtt{S}\to\mathtt{X}} \models \phi] = \sum_{q \in Q} \mathbb{1}\{q \in F\}\mu_{\mathtt{S}\to\mathtt{X}}(q). \tag{5}$$

We can estimate $\mu_{\mathtt{S}\to\mathtt{X}}$ using $\mu_{\mathtt{S}}$ and Equation (3) as follows:

$$\mu_{\mathtt{S}\to\mathtt{X}}(q') = \sum_{q\in Q} \mu_{\mathtt{S}}(q) \frac{1}{|\mathcal{T}^{\mathtt{X}}_{q\to q'}|} \sum_{\tau_i \in \mathcal{T}^{\mathtt{X}}_{q\to q'}} \mathbb{1}\{\delta^*(q,\tau_i) = q'\} \frac{o_{\mathtt{S},q}(\tau_i)}{\iota_{\mathtt{X}}(\tau_i)}, \tag{6}$$

where $\mu_{\mathtt{S}}$ is estimated recursively using μ_0 and Equation (3). Here, notice that $\mathcal{T}^{\mathtt{X}}$ is generated using the initial MDP state distribution of $\mathtt{X}$, denoted by $\iota_{\mathtt{X}}$, and the result we want is with respect to the induced output MDP state distribution of $\mathtt{S}$ for traces that end at the automaton state q (since we are marginalizing over q), denoted by $o_{\mathtt{S},q}$. Therefore, we perform *importance sampling* to account for this distributional mismatch, given by the fraction at the end of the expression. This allows us to treat traces in $\mathcal{T}^{\mathtt{X}}$ as if they were generated starting from the resulting output MDP state distribution of the preceding scenario $\mathtt{S}$. Using this approach, we start from the initial state distribution and compute its evolution throughout the given sequential composition, i.e., $\mu_0 \xrightarrow{\mathtt{S}} \mu_{\mathtt{S}} \xrightarrow{\mathtt{X}} \mu_{\mathtt{S}\to\mathtt{X}}$.

Equation (6) is a valid decomposition of the monolithic SMC query due to the assumed Markovian dynamics of the underlying stochastic process, i.e., it is sufficient to account for the distributional mismatch between the last and the first MDP states of the scenarios $\mathtt{S}$ and $\mathtt{X}$, respectively. In practice, it is crucial to ensure that the output and input MDP state distributions of sequentially composed scenarios like $\mathtt{S}$ and $\mathtt{X}$ overlap. We assume it to be the case, and thus, we only account for the distributional mismatch through importance sampling. As such, we estimate $\iota_{\mathtt{X}}$ and $o_{\mathtt{S}}$ using Gaussian *kernel density estimation* (KDE), an asymptotically valid nonparametric method. KDE performs well in moderate dimensions, whereas it might suffer from the curse of dimensionality in high-dimensional spaces. We expect to apply the proposed approach in moderately sized semantic feature spaces; therefore, for our purposes, KDE suffices.

3.2 Choose Composition

Consider the choose composition $\texttt{choose}(\mathtt{S},\mathtt{X})$. Let $\tau^{\texttt{choose}(\mathtt{S},\mathtt{X})}$, $\tau^{\mathtt{S}}$, and $\tau^{\mathtt{X}}$ denote traces sampled for the composite scenario and the corresponding primitive scenarios. The satisfaction probability can be written as a mixture distribution:

$$\Pr[\tau^{\texttt{choose}(\mathtt{S},\mathtt{X})} \models \phi] = \frac{1}{2}\Pr[\tau^{\mathtt{S}} \models \phi] + \frac{1}{2}\Pr[\tau^{\mathtt{X}} \models \phi]. \tag{7}$$

Intuitively, the composite scenario induces a probabilistic choice over the constituent scenarios, and the overall satisfaction probability is obtained by marginalizing over the selected branch. This expression can be easily extended to n-ary and weighted choose compositions by marginalizing with branch probabilities.

3.3 Shuffle Composition

Finally, consider the shuffle composition $\texttt{shuffle}(\mathtt{S},\mathtt{X})$, which randomly picks a permutation of the given scenarios to sequentially compose. Let $\tau^{\texttt{shuffle}(\mathtt{S},\mathtt{X})}$

denote a trace sampled from it. We can decompose the SMC query as follows:

$$\Pr[\tau^{\texttt{shuffle(S,X)}} \models \phi] = \frac{1}{2}\Pr[\tau^{\texttt{S}\to\texttt{X}} \models \phi] + \frac{1}{2}\Pr[\tau^{\texttt{X}\to\texttt{S}} \models \phi]. \tag{8}$$

This can be trivially extended to n-ary and weighted shuffle compositions. While shuffle composition results in a factorial growth in the number of permutations, we expect the number of shuffled scenarios to remain relatively small.

4 Empirical Evaluation

In this section, we empirically evaluate the proposed framework along two dimensions: *accuracy* and *efficiency*. To evaluate accuracy, we impose a fixed *trace budget* on both our compositional SMC approach and a monolithic SMC baseline, and compare the resulting satisfaction-probability estimates against those obtained by the baseline. To evaluate efficiency, we impose a fixed *time budget*, limiting both approaches to generating traces within the allotted time. We then compare the resulting estimates with their respective counterparts from the fixed-trace-budget experiment, assessing whether each approach can converge to accurate estimates under a limited data budget. This setting is particularly relevant during early-stage system development, when rapid feedback is valuable.

We implemented the proposed framework[1] as a prototype integrated with VerifAI [9], a toolkit for the formal design and analysis of AI-based systems. We use MetaDrive [17] as the simulator and Scenic [11,12] as the scenario description language. We test MetaDrive's built-in expert policy, a neural controller trained to handle general driving scenarios.

We evaluate the framework on seven composite scenarios: (i) `S` → `X`, (ii) `S` → `X` → `S`, (iii) `S` → `O` → `C`, (iv) `C` → `S` → `X` → `S`, (v) `C` → `X` → `S` → `X` → `C`, (vi) `S` → `choose` (`C`, `X`, `O`), and (vii) `S` → `shuffle` (`C`, `X`, `O`). These comprise five sequential compositions of varying lengths and two compositions using `choose` and `shuffle`. We consider four specifications: two safety properties, **2-Stop**, which forbids more than one near-stop event, and **Tollgate**, which forbids remaining below a slow-speed threshold for more than three consecutive steps; and two co-safety properties, **V-Shaped**, which requires the ego to eventually complete a fast–slow–fast speed profile, and **Sustained**, which requires the ego to eventually sustain a large steering input for three consecutive steps. The four specifications are detailed in Appendix A. This yields 28 scenario–specification pairs, each evaluated in both experiments.

4.1 Evaluating Accuracy

In the fixed-trace-budget experiment, we generate 1000 composite scenario traces for the monolithic SMC baseline. For our compositional SMC method, we instead generate 1000 traces for each primitive scenario and reuse these traces across all considered composite scenarios.

[1] Our code is at https://github.com/abhip02/VerifAI/tree/compositional-analysis.

Table 1: Estimated satisfaction probabilities for the **Tollgate** specification under the fixed-trace-budget regime.

Scenario	Monolithic SMC (baseline)	Compositional SMC (ours)	Δ
S → X	0.6750	0.7074	+0.0324
S → X → S	0.6520	0.6898	+0.0378
S → O → C	0.4310	0.4740	+0.0430
C → S → X → S	0.5670	0.6206	+0.0536
C → X → S → X → C	0.3820	0.3819	−0.0001
S → choose (C, X, O)	0.9800	0.9584	−0.0216
S → shuffle (C, X, O)	0.9150	0.8941	−0.0209

Table 1 presents results for the **Tollgate** specification as an example. The compositional estimates closely track the values reported by the monolithic baseline, demonstrating the accuracy of our compositional framework. We observe similar agreement for nearly all remaining scenario–specification pairs, as reported in Table 2 of Appendix B. One notable exception is the **Sustained** specification for the S → shuffle (C, X, O) scenario, for which Δ reaches +0.1162. This gap arises from MDP initial-state correlation discussed in Section 3.1: in the monolithic execution, each subsequent scenario begins from an MDP state the previous scenario left off at, whereas the compositional model samples each subsequent scenario from its own initial MDP state distribution. Although we use importance sampling to account for this distributional mismatch, the quality of the compositional estimate depends directly on the degree of overlap between the MDP state distributions at the interfaces of consecutive scenarios. This highlights that the initial state distributions of primitive scenarios must be chosen carefully to ensure accurate compositional estimates. One way we ensure this in our experiments is by simulating distinct geometries of the same primitive scenario to yield a representative state distribution. For example, we randomly sample four-way intersections on the same map with varying initial velocities and headings. By diversifying these initial conditions, we maximize the likelihood that state distributions overlap across scenario boundaries. Finally, note that the expert policy used in our experiments was not trained to satisfy any of the considered specifications; rather, it serves as a placeholder.

4.2 Evaluating Efficiency

In the fixed-time-budget experiment, both the compositional approach and the monolithic baseline are given a 15-minute wall-clock budget for trace generation. Under this budget, the compositional estimates converge faster toward their respective fixed-trace-budget references than the monolithic baseline, with a mean deviation of 0.029 versus 0.039. This improvement is enabled by the compositional method generating 14.6× more traces on average (ranging from 5.2× to 32.3× more traces). Figure 2 shows a representative scenario–specification pair.

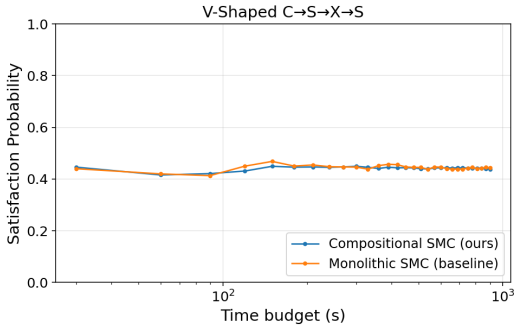


Fig. 2: Fixed-time-budget convergence for the **V-Shaped** specification on the `C` → `S` → `X` → `S` scenario. 753 monolithic and 14871 compositional traces generated within the 15-minute budget.

Within the same wall-clock budget, the compositional method generates 14,871 primitive traces, compared with 753 monolithic traces. Full results are provided in Appendix C.1.

The gap is concentrated among pairs involving `choose`/`shuffle` scenarios, where the per-trace cost is sufficiently high that the 15-minute budget yields as few as 20 monolithic traces. The gap is largest for the ⟨**V-Shaped**, `S` → `shuffle` (`C`, `X`, `O`)⟩ pair, for which the monolithic estimate has an error of 0.275 relative to the reference, compared with 0.014 for the compositional approach. Even when the time budget is extended to 60 minutes, the monolithic baseline still achieves an error of 0.188, substantially higher than the error achieved by the compositional approach with only one quarter of the time budget (15 minutes). Full results for the 60-minute experiment are provided in Appendix C.2.

Finally, we conduct a compute-matched ablation in which the monolithic baseline retains the 15-minute budget, while the compositional approach receives an equal budget for each of the four primitive scenarios (`S`, `X`, `O`, `C`), yielding 225 seconds per primitive. With the sample-count advantage removed, the two approaches are essentially tied, with mean deviations of 0.038 and 0.039, respectively, across the 28 scenario–specification pairs. This result further validates that the primary benefit of our approach comes from sample reuse: a single pool of primitive traces can serve all composite scenarios, whereas the monolithic baseline must regenerate traces independently for each composite scenario. Full results are provided in Appendix C.3.

5 Conclusion

We presented a scenario-based compositional statistical model checking framework for safety and co-safety specifications. By decomposing composite scenarios into primitive scenarios and specifications into sub-specifications, our approach enables independent verification of primitive scenarios whose results are

then composed using importance sampling and kernel density estimation to answer SMC queries about previously unseen composite scenarios. The framework supports sequential, choose, and shuffle compositions. Our empirical evaluation demonstrates that the proposed approach produces accurate estimates while reducing simulation cost through parallelization and trace reuse—a particularly valuable property in early-stage development when rapid feedback is essential.

Acknowledgments. This work was partially supported by the DARPA TIAMAT program, NSF grant 2303564, the California Partners for Advanced Transportation Technology (PATH) Future Mobility Research Center (FMRC), and Nissan through the iCyPhy Center. This work originated as a class project in EECS 219C at UC Berkeley.

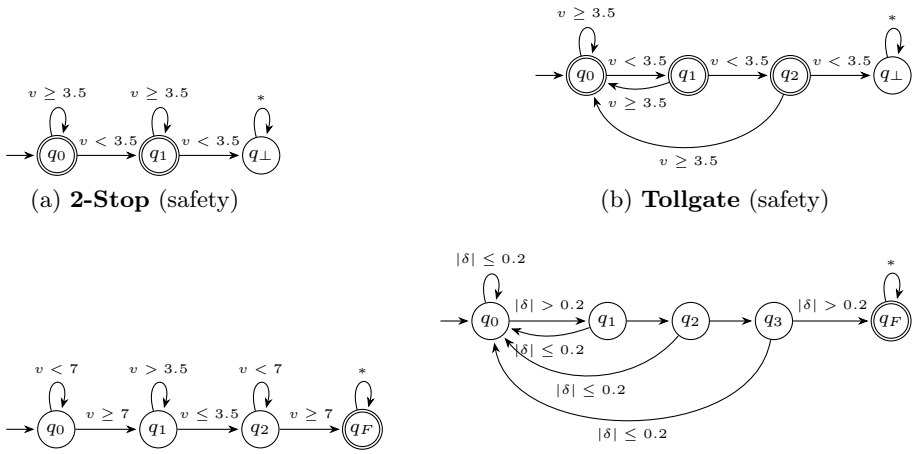
References

1. Abbas, H., Fainekos, G., Sankaranarayanan, S., Ivančić, F., Gupta, A.: Probabilistic temporal logic falsification of cyber-physical systems. ACM Transactions on Embedded Computing Systems (TECS) **12**(2s), 1–30 (2013)
2. Agha, G., Palmskog, K.: A survey of statistical model checking. ACM Transactions on Modeling and Computer Simulation (TOMACS) **28**(1), 1–39 (2018)
3. Chang, K.K.C., Xu, K., Kim, E., Sangiovanni-Vincentelli, A., Seshia, S.A.: Dynamic, multi-objective specification and falsification of autonomous CPS. In: International Conference on Runtime Verification (RV). pp. 40–58. Springer (2024)
4. Chilton, C., Jonsson, B., Kwiatkowska, M.: Compositional assume–guarantee reasoning for input/output component theories. Science of Computer Programming **91**, 115–137 (2014)
5. Clarke, E.M., Zuliani, P.: Statistical model checking for cyber-physical systems. In: International symposium on automated technology for verification and analysis (ATVA). pp. 1–12. Springer (2011)
6. Clarke, E., Long, D., McMillan, K.: Compositional model checking. In: 4th Annual Symposium on Logic in Computer Science (LICS). pp. 353–362 (1989)
7. Cobleigh, J.M., Giannakopoulou, D., Păsăreanu, C.S.: Learning assumptions for compositional verification. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS). pp. 331–346. Springer (2003)
8. Dreossi, T., Donzé, A., Seshia, S.A.: Compositional falsification of cyber-physical systems with machine learning components. Journal of Automated Reasoning **63**(4), 1031–1053 (2019)
9. Dreossi, T., Fremont, D.J., Ghosh, S., Kim, E., Ravanbakhsh, H., Vazquez-Chanlatte, M., Seshia, S.A.: VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems. In: International Conference on Computer Aided Verification (CAV). pp. 432–442. Springer (2019)
10. Duong, H., Shriver, D., Nguyen, T., Dwyer, M.: Compositional neural network verification via assume-guarantee reasoning. Advances in Neural Information Processing Systems (NeurIPS) **38**, 64158–64182 (2025)
11. Fremont, D.J., Dreossi, T., Ghosh, S., Yue, X., Sangiovanni-Vincentelli, A.L., Seshia, S.A.: Scenic: A language for scenario specification and scene generation. In: ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). pp. 63–78 (2019)

12. Fremont, D.J., Kim, E., Dreossi, T., Ghosh, S., Yue, X., Sangiovanni-Vincentelli, A.L., Seshia, S.A.: Scenic: A language for scenario specification and data generation. Machine Learning **112**(10), 3805–3849 (2023)
13. Ivanov, R., Jothimurugan, K., Hsu, S., Vaidya, S., Alur, R., Bastani, O.: Compositional learning and verification of neural network controllers. ACM Transactions on Embedded Computing Systems (TECS) **20**(5s), 1–26 (2021)
14. Junges, S., Spaan, M.T.: Abstraction-refinement for hierarchical probabilistic models. In: International Conference on Computer Aided Verification (CAV). pp. 102–123. Springer (2022)
15. Kloek, T., Van Dijk, H.K.: Bayesian estimates of equation system parameters: an application of integration by monte carlo. Econometrica: Journal of the Econometric Society pp. 1–19 (1978)
16. Legay, A., Delahaye, B., Bensalem, S.: Statistical model checking: An overview. In: International conference on runtime verification (RV). pp. 122–135. Springer (2010)
17. Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., Zhou, B.: MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning. IEEE transactions on pattern analysis and machine intelligence (TPAMI) **45**(3), 3461–3475 (2022)
18. Sen, K., Viswanathan, M., Agha, G.: Statistical model checking of black-box probabilistic systems. In: International Conference on Computer Aided Verification (CAV). pp. 202–215. Springer (2004)
19. Sen, K., Viswanathan, M., Agha, G.: On statistical model checking of stochastic systems. In: International conference on computer aided verification (CAV). pp. 266–280. Springer (2005)
20. Seshia, S.A., Sadigh, D., Sastry, S.S.: Toward verified artificial intelligence. Communications of the ACM **65**(7), 46–55 (2022)
21. Vin, E., Miller, K.A., Incer, I., Seshia, S.A., Fremont, D.J.: ScenicProver: A framework for compositional probabilistic verification of learning-enabled systems. arXiv preprint arXiv:2511.02164 (2025)
22. Watanabe, K., Eberhart, C., Asada, K., Hasuo, I.: Compositional probabilistic model checking with string diagrams of MDPs. In: International Conference on Computer Aided Verification (CAV). pp. 40–61. Springer (2023)
23. Watson, C., Alur, R., Gopinath, D., Mangal, R., Păsăreanu, C.S.: Scenario-based compositional verification of autonomous systems with neural perception. In: International Symposium on AI Verification (SAIV). pp. 3–28. Springer (2025)
24. Yalcinkaya, B., Torfah, H., Fremont, D.J., Seshia, S.A.: Compositional simulation-based analysis of ai-based autonomous systems for markovian specifications. In: International Conference on Runtime Verification (RV). pp. 191–212. Springer (2023)
25. Younes, H.L., Kwiatkowska, M., Norman, G., Parker, D.: Numerical vs. statistical probabilistic model checking. International Journal on Software Tools for Technology Transfer **8**(3), 216–228 (2006)
26. Younes, H.L., Simmons, R.G.: Statistical probabilistic model checking with a focus on time-bounded properties. Information and Computation **204**(9), 1368–1409 (2006)
27. Zuliani, P., Platzer, A., Clarke, E.M.: Bayesian statistical model checking with application to stateflow/simulink verification. Formal Methods in System Design **43**(2), 338–367 (2013)

A Formal Specifications

Figure 3 illustrates the four specifications used in the experiments.



(a) **2-Stop** (safety)

(b) **Tollgate** (safety)

(c) **V-Shaped** (co-safety)

(d) **Sustained** steering (co-safety)

Fig. 3: Safety and co-safety specifications used in the experiments. Here v is the vehicle speed (m/s) and δ the steering angle; doubly circled states are accepting, q_0 is the initial state, and $*$ marks a self-loop on any symbol.

B Full Accuracy Results

This section presents the full experimental results corresponding to Section 4.1.

Table 2: Estimated satisfaction probabilities for the monolithic baseline and our compositional method under the fixed-budget regime described in Section 4.1. Δ denotes the compositional estimate minus the monolithic estimate.

Scenario	Monolithic	Compositional	Δ
2-Stop (safety)			
S → X	0.6650	0.6712	+0.0062
S → X → S	0.6000	0.6700	+0.0700
S → O → C	0.3890	0.4511	+0.0621
C → S → X → S	0.5380	0.6156	+0.0776
C → X → S → X → C	0.3550	0.3849	+0.0299
S → choose (C, X, O)	0.9770	0.9511	−0.0259
S → shuffle (C, X, O)	0.8910	0.8775	−0.0135
Tollgate (safety)			
S → X	0.6750	0.7074	+0.0324
S → X → S	0.6520	0.6898	+0.0378
S → O → C	0.4310	0.4740	+0.0430
C → S → X → S	0.5670	0.6206	+0.0536
C → X → S → X → C	0.3820	0.3819	−0.0001
S → choose (C, X, O)	0.9800	0.9584	−0.0216
S → shuffle (C, X, O)	0.9150	0.8941	−0.0209
V-Shaped (co-safety)			
S → X	0.3580	0.3327	−0.0253
S → X → S	0.3930	0.3524	−0.0406
S → O → C	0.5570	0.5784	+0.0214
C → S → X → S	0.4720	0.4194	−0.0526
C → X → S → X → C	0.6150	0.6577	+0.0427
S → choose (C, X, O)	0.0130	0.0415	+0.0285
S → shuffle (C, X, O)	0.3250	0.3101	−0.0149
Sustained (co-safety)			
S → X	0.5680	0.5305	−0.0375
S → X → S	0.6280	0.5479	−0.0801
S → O → C	0.9630	0.9960	+0.0330
C → S → X → S	0.7180	0.6612	−0.0568
C → X → S → X → C	0.8420	0.8729	+0.0309
S → choose (C, X, O)	0.2550	0.2377	−0.0173
S → shuffle (C, X, O)	0.3100	0.4262	+0.1162

C Full Efficiency Results

This section presents the complete experimental results corresponding to Section 4.2. We evaluate efficiency under three settings: (i) both the compositional approach and the monolithic baseline are given a 15-minute wall-clock budget for trace generation (Appendix C.1), (ii) both approaches are given a 60-minute wall-clock budget for trace generation (Appendix C.2), and (iii) a compute-matched ablation in which the monolithic baseline retains the 15-minute budget, while the compositional approach receives an equal budget for each of the four primitive scenarios (`S`, `X`, `O`, `C`), yielding 225 seconds per primitive (Appendix C.3).

C.1 15-Minute Time-Budget Experiments

Table 3 reports the compositional and monolithic estimates under a fixed 15-minute time budget for all 28 specification–scenario cells, and Figures 4 to 7 show how these estimates converge as the budget grows.

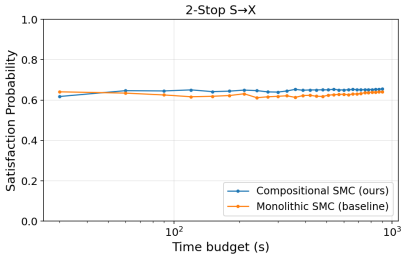
Table 3: 15-minute time-budget results. ρ is the satisfaction probability estimate; N is the number of traces generated; Δ denotes the compositional estimate minus the monolithic estimate.

	Monolithic		Compositional		
Scenario	N	ρ	N	ρ	Δ
Two-stop (safety)					
`S` → `X`	2347	0.6412	12262	0.6558	+0.0146
`S` → `X` → `S`	1618	0.6057	12262	0.6533	+0.0476
`S` → `O` → `C`	788	0.3820	12335	0.4410	+0.0590
`C` → `S` → `X` → `S`	753	0.5618	14871	0.5892	+0.0275
`C` → `X` → `S` → `X` → `C`	460	0.3217	14871	0.3547	+0.0330
`S` → `choose` (`C`, `X`, `O`)	49	1.0000	312	0.9676	−0.0324
`S` → `shuffle` (`C`, `X`, `O`)	20	0.9500	312	0.9062	−0.0438
Tollgate (safety)					
`S` → `X`	2347	0.6860	12262	0.6792	−0.0068
`S` → `X` → `S`	1618	0.6508	12262	0.6770	+0.0262
`S` → `O` → `C`	788	0.4226	12335	0.4638	+0.0412
`C` → `S` → `X` → `S`	753	0.6069	14871	0.6195	+0.0126
`C` → `X` → `S` → `X` → `C`	460	0.3630	14871	0.3922	+0.0292
`S` → `choose` (`C`, `X`, `O`)	49	1.0000	312	0.9769	−0.0231
`S` → `shuffle` (`C`, `X`, `O`)	20	0.9500	312	0.9322	−0.0178
V-shaped (co-safety, reported as $1-\rho$)					
`S` → `X`	2347	0.3775	12262	0.3467	−0.0308

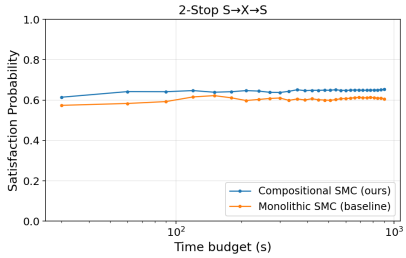
continued on next page

Table 3 continued

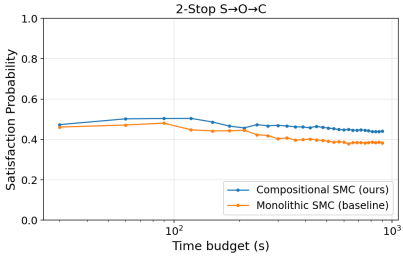
Scenario	Monolithic N	Monolithic ρ	Compositional N	Compositional ρ	Δ
`S` → `X` → `S`	1618	0.4017	12262	0.3689	−0.0328
`S` → `O` → `C`	788	0.5799	12335	0.5860	+0.0061
`C` → `S` → `X` → `S`	753	0.4449	14871	0.4380	−0.0069
`C` → `X` → `S` → `X` → `C`	460	0.6783	14871	0.6778	−0.0005
`S` → `choose` (`C`, `X`, `O`)	49	0.0204	312	0.0206	+0.0002
`S` → `shuffle` (`C`, `X`, `O`)	20	0.6000	312	0.2964	−0.3036
Sustained (co-safety, reported as $1-\rho$)					
`S` → `X`	2347	0.5684	12262	0.4960	−0.0724
`S` → `X` → `S`	1618	0.6193	12262	0.5167	−0.1025
`S` → `O` → `C`	788	0.9657	12335	0.9954	+0.0296
`C` → `S` → `X` → `S`	753	0.6866	14871	0.6465	−0.0401
`C` → `X` → `S` → `X` → `C`	460	0.8565	14871	0.8627	+0.0062
`S` → `choose` (`C`, `X`, `O`)	49	0.2041	216	0.0813	−0.1228
`S` → `shuffle` (`C`, `X`, `O`)	20	0.5500	216	0.2439	−0.3061



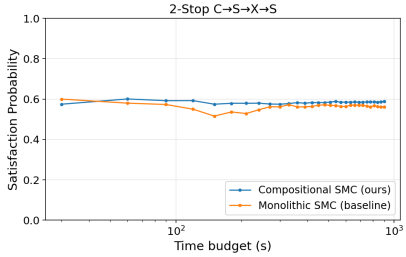
(a) `S` → `X` (N_m=2347, N_c=12262)



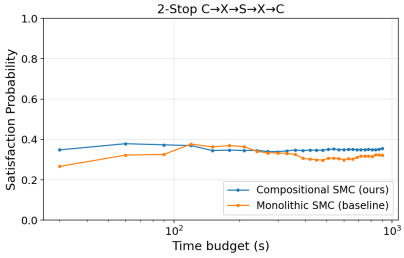
(b) `S` → `X` → `S` (N_m=1618, N_c=12262)



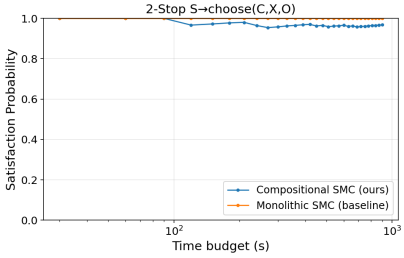
(c) `S` → `O` → `C` (N_m=788, N_c=12335)



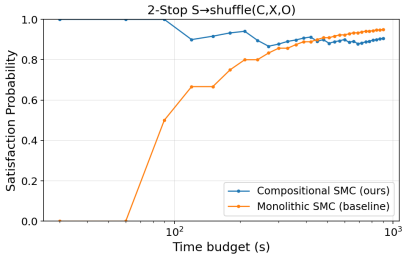
(d) `C` → `S` → `X` → `S` (N_m=753, N_c=14871)



(e) `C` → `X` → `S` → `X` → `C` (N_m=460, N_c=14871)

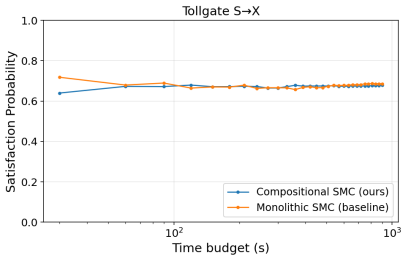


(f) `S` → `choose` (`C`, `X`, `O`) (N_m=49, N_c=312)

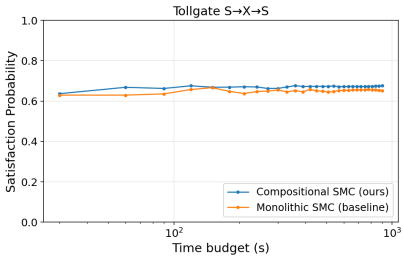


(g) `S` → `shuffle` (`C`, `X`, `O`) (N_m=20, N_c=312)

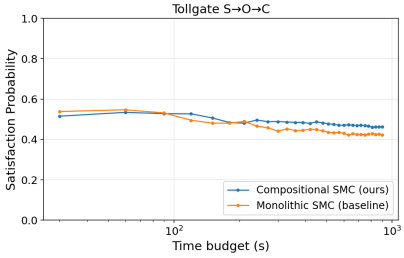
Fig. 4: 15-minute time-budget results for the **2-Stop** specification. N_m and N_c are the number of monolithic and compositional primitive traces generated within the budget.



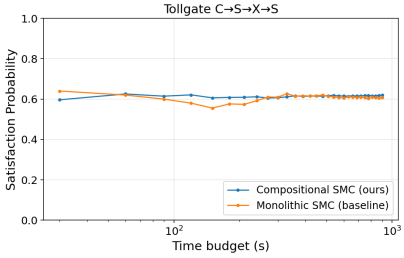
(a) `S` → `X` (N_m=2347, N_c=12262)



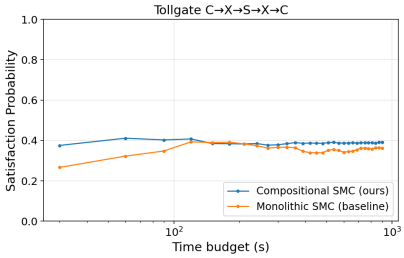
(b) `S` → `X` → `S` (N_m=1618, N_c=12262)



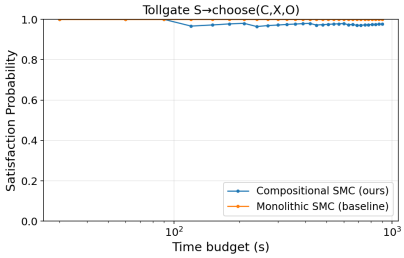
(c) `S` → `O` → `C` (N_m=788, N_c=12335)



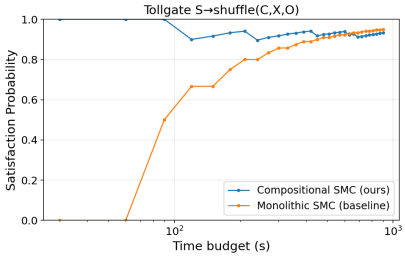
(d) `C` → `S` → `X` → `S` (N_m=753, N_c=14871)



(e) `C` → `X` → `S` → `X` → `C` (N_m=460, N_c=14871)

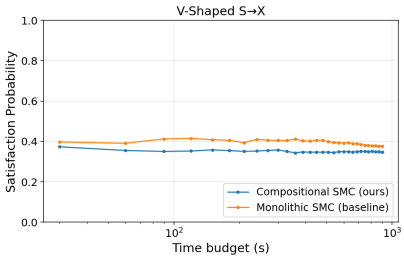


(f) `S` → `choose` (`C`, `X`, `O`) (N_m=49, N_c=312)

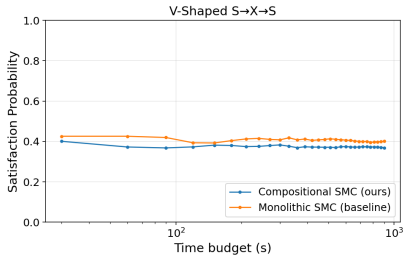


(g) `S` → `shuffle` (`C`, `X`, `O`) (N_m=20, N_c=312)

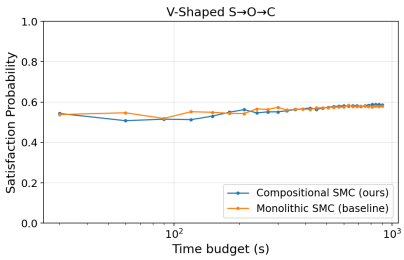
Fig. 5: 15-minute time-budget results for the **Tollgate** specifications. N_m and N_c are the number of monolithic and compositional primitive traces generated within the budget.



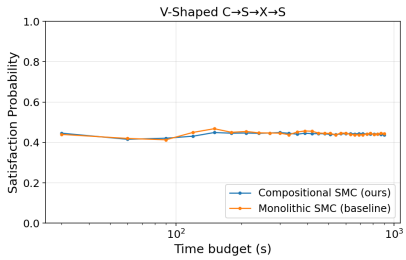
(a) `S` → `X` (N_m=2347, N_c=12262)



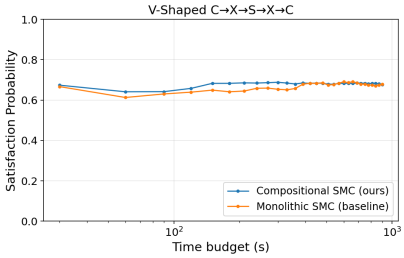
(b) `S` → `X` → `S` (N_m=1618, N_c=12262)



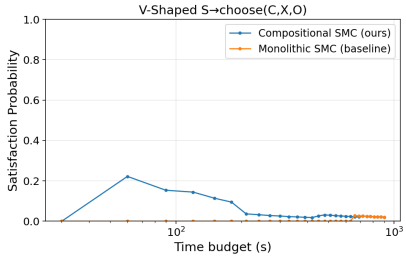
(c) `S` → `O` → `C` (N_m=788, N_c=12335)



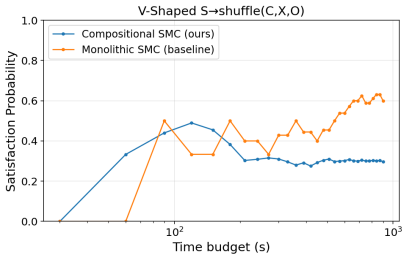
(d) `C` → `S` → `X` → `S` (N_m=753, N_c=14871)



(e) `C` → `X` → `S` → `X` → `C` (N_m=460, N_c=14871)

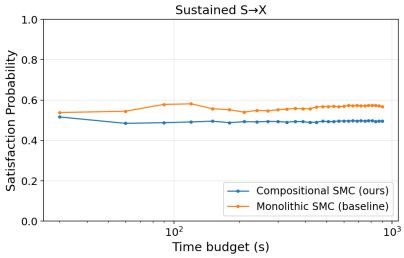


(f) `S` → `choose` (`C`, `X`, `O`) (N_m=49, N_c=312)

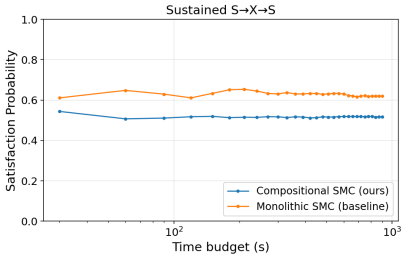


(g) `S` → `shuffle` (`C`, `X`, `O`) (N_m=20, N_c=312)

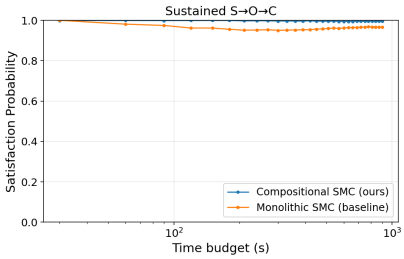
Fig. 6: 15-minute time-budget results for the **V-Shaped** specification. N_m and N_c are the number of monolithic and compositional primitive traces generated within the budget.



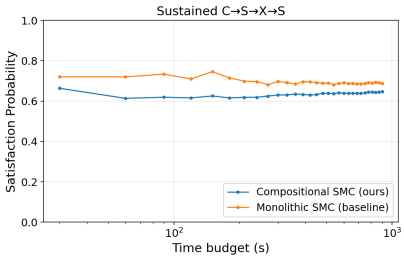
(a) `S` → `X` (N_m=2347, N_c=12262)



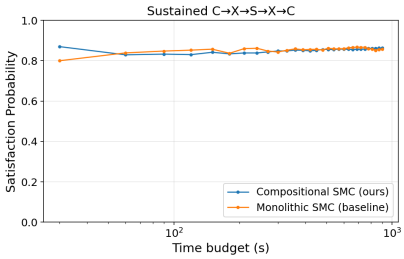
(b) `S` → `X` → `S` (N_m=1618, N_c=12262)



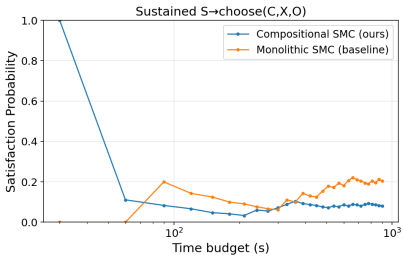
(c) `S` → `O` → `C` (N_m=788, N_c=12335)



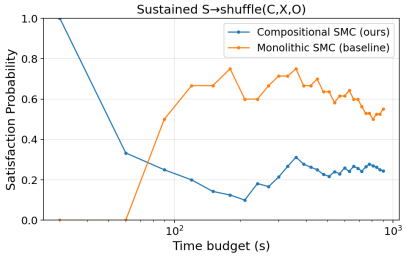
(d) `C` → `S` → `X` → `S` (N_m=753, N_c=14871)



(e) `C` → `X` → `S` → `X` → `C` (N_m=460, N_c=14871)



(f) `S` → `choose` (`C`, `X`, `O`) (N_m=49, N_c=216)



(g) `S` → `shuffle` (`C`, `X`, `O`) (N_m=20, N_c=216)

Fig. 7: 15-minute time-budget results for the **Sustained** specification. N_m and N_c are the number of monolithic and compositional primitive traces generated within the budget.

As described in Section 4.2, the monolithic estimate deviates from each method's own fixed-trace-budget reference (Table 2) by 0.039 against 0.029 for the compositional one. The gap is concentrated among scenarios involving `choose` and `shuffle` (0.089 vs. 0.059), where the 15-minute monolithic budget buys as few as 20 traces.

C.2 60-Minute Time-Budget Experiments

To obtain a sharper monolithic baseline for the sample-starved `choose`/`shuffle` scenarios, we increase the time budget to 60 minutes. Table 4 reports the resulting estimates and Figure 8 shows their convergence.

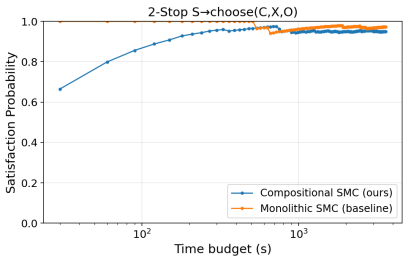
Table 4: 60-minute time-budget results on the `choose`/`shuffle` scenarios. ρ is the satisfaction probability estimate; N is the number of traces generated; Δ denotes the compositional estimate minus the monolithic estimate.

	Monolithic		Compositional		
Scenario	N	ρ	N	ρ	Δ
Two-stop (safety)					
`S` → `choose` (`C`, `X`, `O`)	188	0.9734	1161	0.9503	−0.0231
`S` → `shuffle` (`C`, `X`, `O`)	76	0.9211	1161	0.8795	−0.0415
Tollgate (safety)					
`S` → `choose` (`C`, `X`, `O`)	188	0.9787	1161	0.9568	−0.0219
`S` → `shuffle` (`C`, `X`, `O`)	76	0.9211	1161	0.8964	−0.0246
V-shaped (co-safety, reported as $1-\rho$)					

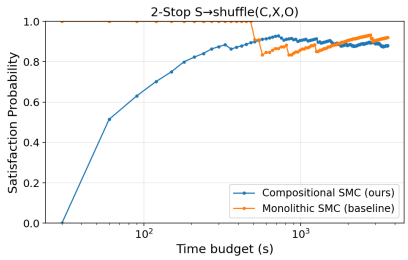
continued on next page

Table 4 continued

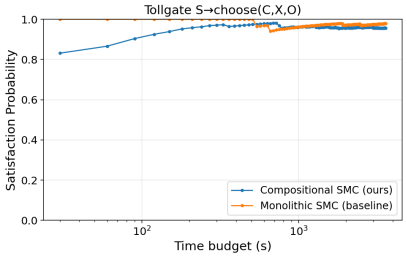
Scenario	Monolithic N	Monolithic ρ	Compositional N	Compositional ρ	Δ
`S` → `choose` (`C`, `X`, `O`)	188	0.0532	1161	0.0509	−0.0023
`S` → `shuffle` (`C`, `X`, `O`)	76	0.5132	1161	0.3490	−0.1642
Sustained steering (co-safety, reported as $1-\rho$)					
`S` → `choose` (`C`, `X`, `O`)	188	0.2234	816	0.1229	−0.1005
`S` → `shuffle` (`C`, `X`, `O`)	76	0.5132	816	0.3665	−0.1467



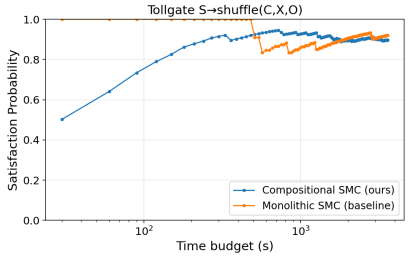
(a) **2-Stop** on `S` → `choose` (`C`, `X`, `O`) (N_m=188, N_c=1161)



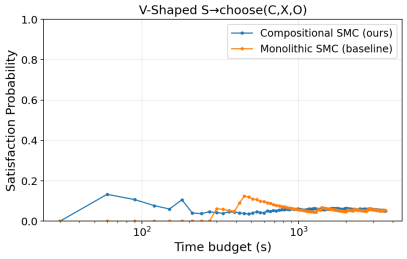
(b) **2-Stop** on `S` → `shuffle` (`C`, `X`, `O`) (N_m=76, N_c=1161)



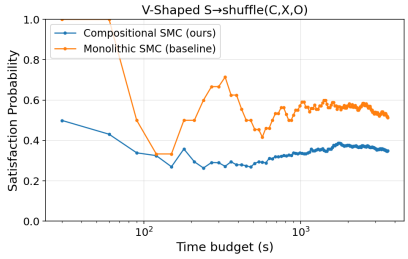
(c) **Tollgate** on `S` → `choose` (`C`, `X`, `O`) (N_m=188, N_c=1161)



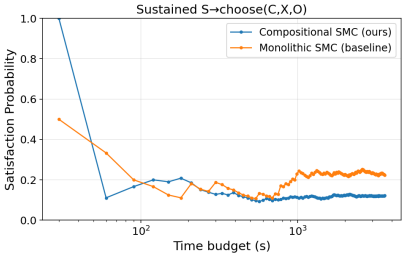
(d) **Tollgate** on `S` → `shuffle` (`C`, `X`, `O`) (N_m=76, N_c=1161)



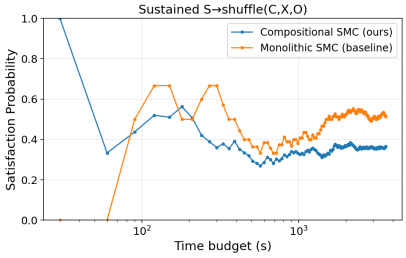
(e) **V-shaped** on `S` → `choose` (`C`, `X`, `O`) (N_m=188, N_c=1161)



(f) **V-shaped** on `S` → `shuffle` (`C`, `X`, `O`) (N_m=76, N_c=1161)



(g) **Sustained** on `S` → `choose` (`C`, `X`, `O`) (N_m=188, N_c=816)



(h) **Sustained** on `S` → `shuffle` (`C`, `X`, `O`) (N_m=76, N_c=816)

Fig. 8: 60-minute time-budget results on the `choose`/`shuffle` scenarios. Figures (a)–(d) are the two safety specifications (**2-Stop**, **Tollgate**). Figures (e)–(h) are the two co-safety specifications (**V-shaped**, **Sustained**), reported as $1 - \rho$. N_m and N_c are the number of monolithic and compositional primitive traces generated within the budget.

With four times the budget, the monolithic estimate becomes more accurate but still does not catch up. Averaged across the eight scenario–specification pairs, its deviation from the reference decreases from 0.089 at 15 minutes to 0.063, whereas the compositional deviation drops from 0.059 to 0.029, leaving the monolithic estimate roughly twice as far from the reference. The remaining gap is concentrated in the two co-safety `shuffle` pairs: even at $N_m = 76$, the monolithic estimate remains 0.188 (**V-Shaped**) and 0.203 (**Sustained**) from its reference, compared with 0.039 and 0.060, respectively, for the compositional method. For **V-Shaped** `shuffle`, whose two reference values differ by only 0.015, this provides a clear convergence comparison: the compositional estimate is already within 0.039 of the shared reference value, while the 60-minute monolithic run remains 0.188 away.

C.3 Compute-Matched Ablation

A natural concern is whether the compositional method's advantage is genuine or merely a consequence of sample count: within a fixed wall-clock window it generates many short primitive traces, whereas the monolithic baseline generates only a few long ones. To control for this, we level the comparison by giving both methods the same total simulation budget rather than the same wall-clock time, charging the compositional method only the cost of generating a single monolithic scenario and asking whether it still recovers the same estimates.

Each primitive pool P_i $(i = 1, \ldots, k)$ contains N_i traces generated within budget T, equal to the budget for one monolithic scenario. Splitting the same budget evenly across the k primitives allocates T/k to each. Assuming a roughly constant per-trace cost, this corresponds to the first $\lfloor N_i/k \rfloor$ traces in each pool. We therefore truncate every pool to this prefix, preserving a total budget of T while leaving the estimator unchanged.

Table 5: Compute-matched ablation. ρ is the satisfaction probability estimate; N is the number of traces generated; Δ denotes the compositional estimate minus the monolithic estimate.

	Monolithic		Compositional (matched)		
Scenario	N	ρ	N	ρ	Δ
Two-stop (safety)					
S → X	2347	0.6412	3064	0.6482	+0.0070
S → X → S	1618	0.6057	3064	0.6459	+0.0402
S → O → C	788	0.3820	3082	0.4617	+0.0797
C → S → X → S	753	0.5618	3716	0.5781	+0.0163
C → X → S → X → C	460	0.3217	3716	0.3422	+0.0205
S → choose (C, X, O)	49	1.0000	77	0.9623	−0.0377
S → shuffle (C, X, O)	20	0.9500	77	0.8905	−0.0595
Tollgate (safety)					
S → X	2347	0.6860	3064	0.6736	−0.0124
S → X → S	1618	0.6508	3064	0.6714	+0.0206
S → O → C	788	0.4226	3082	0.4837	+0.0611
C → S → X → S	753	0.6069	3716	0.6086	+0.0017
C → X → S → X → C	460	0.3630	3716	0.3798	+0.0168
S → choose (C, X, O)	49	1.0000	77	0.9623	−0.0377
S → shuffle (C, X, O)	20	0.9500	77	0.8905	−0.0595
V-shaped (co-safety, reported as $1-\rho$)					
S → X	2347	0.3775	3064	0.3510	−0.0265
S → X → S	1618	0.4017	3064	0.3755	−0.0262
S → O → C	788	0.5799	3082	0.5580	−0.0219
C → S → X → S	753	0.4449	3716	0.4483	+0.0034
C → X → S → X → C	460	0.6783	3716	0.6895	+0.0112
S → choose (C, X, O)	49	0.0204	77	0.0346	+0.0142
S → shuffle (C, X, O)	20	0.6000	77	0.3045	−0.2955
Sustained (co-safety, reported as $1-\rho$)					
S → X	2347	0.5684	3064	0.4922	−0.0762
S → X → S	1618	0.6193	3064	0.5140	−0.1053
S → O → C	788	0.9657	3082	0.9975	+0.0318
C → S → X → S	753	0.6866	3716	0.6191	−0.0675
C → X → S → X → C	460	0.8565	3716	0.8389	−0.0176
S → choose (C, X, O)	49	0.2041	53	0.0333	−0.1708
S → shuffle (C, X, O)	20	0.5500	53	0.1000	−0.4500

With the budget split evenly across the k primitive pools (here, $k = 4$, yielding 225 seconds per pool), the compositional advantage in raw sample count

disappears, yet its accuracy remains essentially unchanged. Relative to their respective 1,000-trace references (Table 2), the compositional and monolithic estimates achieve nearly identical mean deviations across the 28 pairs: 0.038 and 0.039, respectively. Thus, the compositional method retains its accuracy without relying on additional traces, while a single truncated primitive pool can still be reused across all composite scenarios, unlike the monolithic baseline, which must regenerate traces from scratch for each scenario.